

AgriSmart AI: Project Blueprint

Comprehensive System Architecture & Repository Structure for SIH-2026

1. System Architecture Overview

The AgriSmart AI project is a modular, two-stage system designed to operate locally. It combines a highly accurate Computer Vision pipeline for disease identification with an offline Retrieval-Augmented Generation (RAG) pipeline for scientifically grounded farmer advisory.

Phase 1: Computer Vision (Disease Detection)

- **Input:** Raw field image (often cluttered) uploaded by the user.
- **Step A (YOLO Object Detection):** Locates the leaf within the image and draws a bounding box to crop out background noise.
- **Step B (Classification Backbone):** A CNN/ViT model (e.g., EfficientNet) analyzes the cropped leaf and outputs a disease class (e.g., "Tomato Early Blight") and confidence score.

Phase 2: RAG Pipeline (GenAI Advisor)

- **Knowledge Base:** Digitized agricultural textbooks formatted as JSON, chunked, and stored in a local Vector DB (e.g., ChromaDB or FAISS) using dense embeddings (e.g., all-MiniLM-L6-v2).
- **Retrieval:** The disease class predicted by Phase 1 triggers a semantic search in the Vector DB to pull the exact treatment protocols and organic/chemical remedies.
- **Generation:** A local LLM (e.g., Llama-3-8B or Mistral via Ollama) processes the retrieved textbook context and formulates a grounded, conversational response for the farmer.

2. Recommended Directory Structure (GitHub Submission)

This structure ensures compliance with the SIH-2026 hackathon requirements, providing clear separation between model weights, application logic, and the RAG document database.

```
agrismart-ai/  
├── README.md  
└── instructions)
```

```
# Entry point (setup, metrics, run
```

```

├─ requirements.txt          # Python dependencies (FastAPI, PyTorch,
Langchain, etc.)
├─ .env.example              # Environment variables (DB paths, API keys
for weather)
|
├─ data/                     # Local datasets (not uploaded to git,
use .gitignore)
|   ├─ raw_images/           # PlantVillage and PlantDoc field images
|   ├─ textbook_raw/         # PDF/OCR scans of agricultural textbooks
|   └─ structured_db.json    # Cleaned textbook data mapped to disease
classes
|
├─ src/                      # Main source code directory
|   ├─ cv_pipeline/          # Phase 1: Computer Vision
|   |   ├─ train_yolo.py      # Script for training bounding box model
|   |   ├─ train_classifier.py # Script for training disease classifier
|   |   └─ predict_vision.py  # Inference script for the core mandatory
task
|   |
|   └─ rag_pipeline/         # Phase 2: Knowledge Retrieval
|       └─ ingest_books.py    # Script to chunk and embed textbook JSON to
Vector DB
|   |
|   |   ├─ retriever.py       # FAISS/ChromaDB search logic
|   |   └─ llm_agent.py       # Local LLM integration (Ollama/llama.cpp)
|   |
|   └─ api/                  # Web Integration
|       ├─ main.py            # FastAPI backend routing inputs to models
|       └─ schemas.py         # Pydantic models for API requests/responses
|
├─ app/                      # Frontend Interface
|   ├─ streamlit_app.py       # (Option A) Rapid prototyping dashboard
|   └─ nextjs_frontend/       # (Option B) Polished UI with conversational
chat
|
├─ vector_store/             # ChromaDB / FAISS embedded database files
|
├─ models/                   # Saved model weights
|   ├─ yolo_leaf_detector.pt  # Cropping model weights
|   └─ disease_classifier.h5  # Final trained CV backbone
|
└─ report/                   # Mandatory hackathon documentation
    └─ model_report.pdf       # 1-page report detailing Macro-F1, ROC, &
architecture

```

3. Execution Flow (The "Agentic" Loop)

When the user submits an image via the frontend:

1. The image is sent to `predict_vision.py`.
2. The model returns: `{"disease": "Potato Late Blight", "confidence": 0.94}`.
3. This triggers the RAG retriever (`retriever.py`) to search the vector store for "Potato Late Blight treatments and precautions".
4. (Optional Bonus) The backend fetches current weather conditions via an external API.
5. The retrieved textbook text, weather data, and disease label are injected into an LLM prompt (`llm_agent.py`).
6. The LLM generates a cohesive, plain-language advisory message (e.g., "The model detected Late Blight. Because it will rain tomorrow, delay irrigation and apply [Textbook Fungicide] today.").

4. Submission Checklist for SIH-2026

- **Core Task:** Can the model accept a single image and output a class + confidence? (Required)
- **Evaluation Metrics:** Did you report Macro-F1 and a confusion matrix on the held-out set?
- **Reproducibility:** Can the judges clone the repo, install requirements, and run inference in under 10 minutes?
- **GenAI Bonus:** Does the RAG output explicitly cite the localized textbook data to avoid hallucinations?
- **Video Demo:** 3-5 minute video showcasing the upload, the vision detection, and the LLM's explanation.